

EXPERIMENT - 5

Title

Create a Food Delivery Website where Users can Order Food from a Particular Restaurant using Node.js for Handling HTTP Requests and Responses

Aim

To develop a simple **Food Delivery Website** using Node.js that allows users to:

- View a restaurant and food menu
 - Place an order
 - Receive an order confirmation by handling HTTP requests and responses.
-

Prerequisites

- Knowledge of JavaScript
 - Basic understanding of Node.js HTTP module
 - Completion of Experiments 1 to 4
-

Software Requirements

- Operating System: Windows / Linux / macOS
- Runtime Environment: Node.js (LTS)
- Code Editor: Visual Studio Code
- Browser: Google Chrome

Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB

Theory

A **Food Delivery Website** allows users to browse restaurants, select food items, and place orders.

In this experiment:

- Node.js **HTTP module** is used to create a server
- GET request displays restaurant menu
- POST request handles food order
- Request data is processed and response is generated dynamically

This experiment demonstrates **real-time HTTP request and response handling** without using any frameworks.

Algorithm

1. Import required Node.js modules.
2. Create an HTTP server.
3. Display restaurant and menu items.
4. Accept food order using HTML form.
5. Process order using POST request.
6. Display order confirmation.
7. Run and test the application.

Project Folder Structure

```
fsd_exp5/  
├── app.js  
└── menu.html
```

Source Code

menu.html

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Food Delivery</title>  
</head>  
<body>  
  <h1>Spice Hub Restaurant</h1>  
  <h3>Menu</h3>  
  
  <form method="POST" action="/order">  
  
    <input type="radio" name="food" value="Biryani" required /> Biryani -  
&#8377; 200<br>  
  
    <input type="radio" name="food" value="Fried Rice" /> Fried Rice -  
&#8377; 150<br>  
  
    <input type="radio" name="food" value="Paneer Curry" /> Paneer Curry -  
&#8377; 180<br><br>  
  
    <label>Customer Name:</label>  
    <input type="text" name="customer" required /><br><br>  
  
    <button type="submit">Place Order</button>  
  </form>  
</body>  
</html>
```

app.js

```
const http = require('http');
const fs = require('fs');
const querystring = require('querystring');

const server = http.createServer((req, res) => {

  // Show Menu Page
  if (req.method === 'GET' && req.url === '/') {
    fs.readFile('menu.html', (err, data) => {
      if (err) {
        res.writeHead(500);
        res.end('Error loading page');
      } else {
        res.writeHead(200, { 'Content-Type': 'text/html' });
        res.end(data);
      }
    });
  }

  // Handle Order Submission
  else if (req.method === 'POST' && req.url === '/order') {
    let body = '';

    req.on('data', chunk => {
      body += chunk.toString();
    });

    req.on('end', () => {
      const formData = querystring.parse(body);

      res.writeHead(200, { 'Content-Type': 'text/html' });
      res.write(`
        <h2>Order Confirmed</h2>
        <p>Thank you <b>${formData.customer}</b></p>
        <p>Your order for <b>${formData.food}</b> has been placed
successfully.</p>
        <a href="/">Order Again</a>
      `);
      res.end();
    });
  }
});

server.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

Explanation of Code

- `http` → creates server
 - `fs.readFile()` → loads HTML menu page
 - `querystring.parse()` → reads form data
 - GET request → displays menu
 - POST request → processes food order
 - Dynamic response sent to browser
-

Execution Steps

1. Open terminal in project folder
 2. Run the application:
 3. `node app.js`
 4. Open browser:
 5. `http://localhost:3000`
-

Sample Input

Customer Name: Ravi

Food Item: Biryani

Sample Output

Order Confirmed

Thank you Ravi

Your order for Biryani has been placed successfully.

Expected Result

The Food Delivery Website successfully displays menu items, accepts orders, and confirms the order using Node.js HTTP request and response handling.

Result

Thus, a Food Delivery Website using Node.js was successfully developed to handle HTTP requests and responses.

Viva Voce Questions

1. How does Node.js handle HTTP requests?
 2. Difference between GET and POST?
 3. What is querystring module used for?
 4. How is this different from Express.js?
 5. Can multiple restaurants be added?
 6. How can payment integration be done?
-

Conclusion

This experiment demonstrates the practical usage of Node.js HTTP module to build a real-world application that handles client requests and server responses efficiently.